Autenticación y autorización

Ingeniería de Sistemas

Universidad Austral — 2026

Diseñen el *login* del Snippet Searcher

Tienen dos minutos

La propuesta de siempre

```sql
CREATE TABLE users (
  email    TEXT PRIMARY KEY,
  password TEXT
);
```

```
fun login(request: LoginRequest): Response {
    val user = users.findByEmail(request.email) ?: return notFound("Usuario no existe")
    return if (user.password == request.password) ok() else unauthorized("Contraseña incorrecta")
}
```

Parece simple. **Lo dejamos acá y volvemos en un rato.**

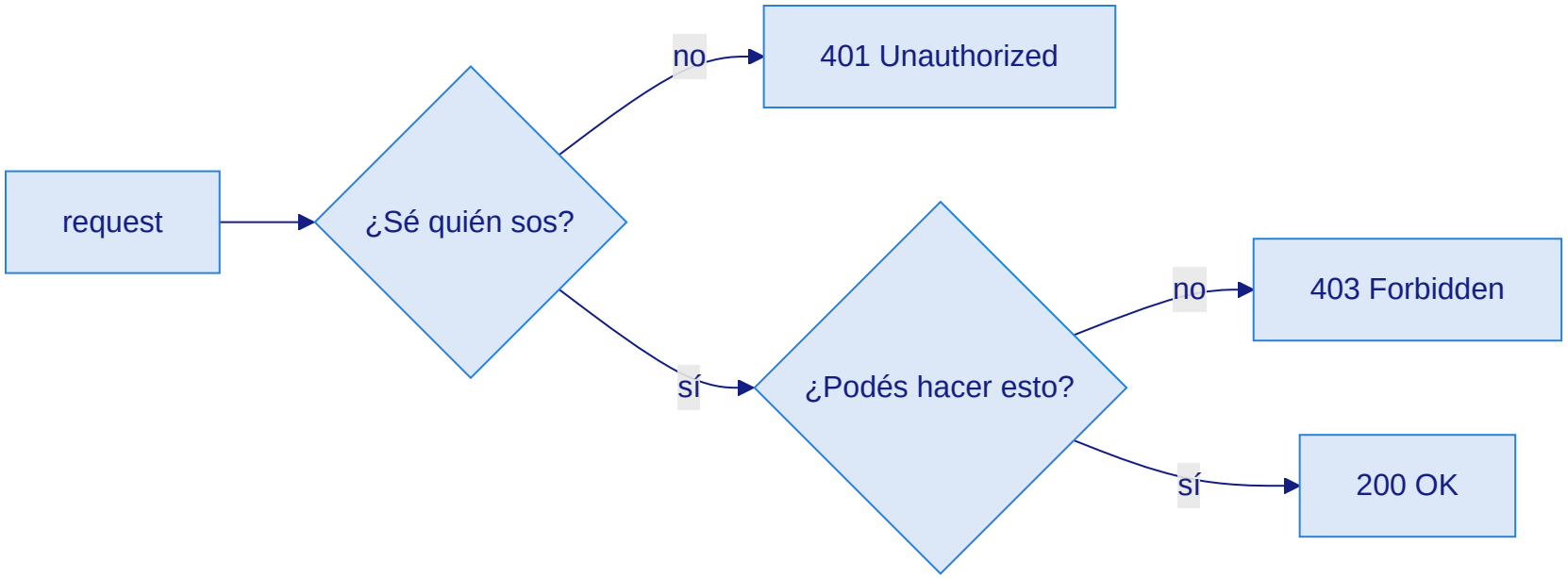
Dos preguntas distintas

Autenticación vs autorización

	Autenticación (*authn*)	**Autorización** (*authz*)
Pregunta	¿Quién sos?	¿Qué podés hacer?
Verifica	una identidad	un permiso sobre un recurso
Si falla	**401**	**403**
Ejemplo	*login* con email y contraseña	sólo el dueño borra un *snippet*

Siempre en ese orden: no se puede autorizar a alguien que no se identificó.

HTTP lo dice con sus códigos



`401 Unauthorized` es un nombre histórico: en realidad significa **no autenticado**.

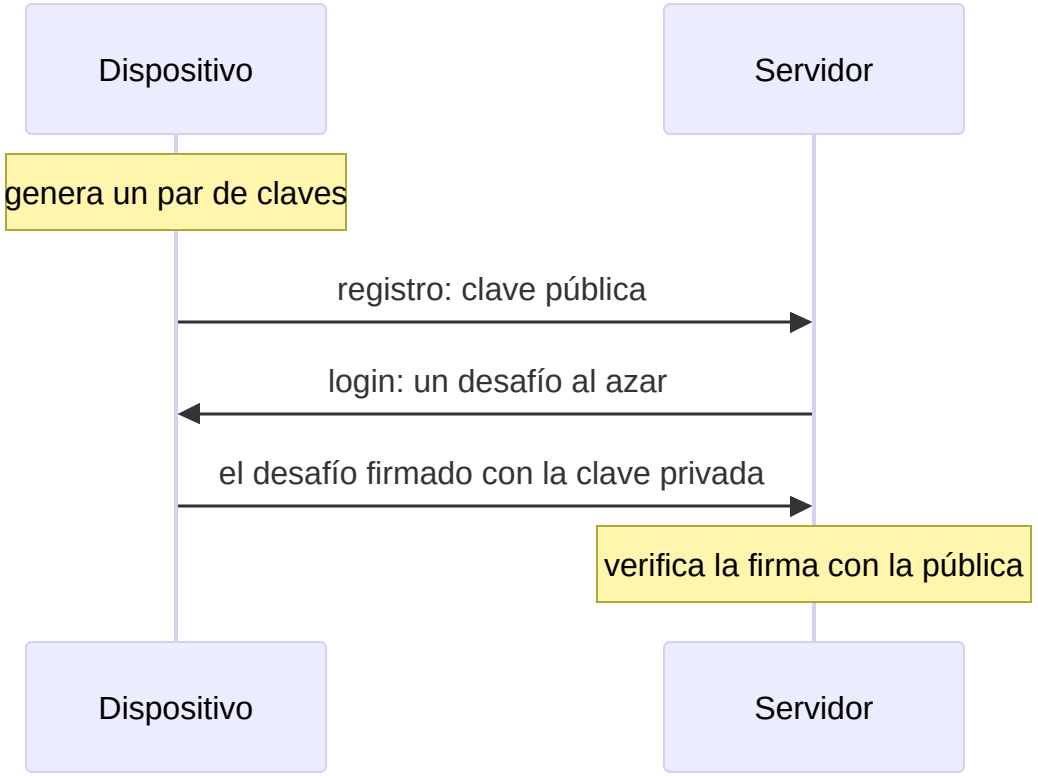
¿Cómo se autentica alguien?

Factores de autenticación

- **Algo que sabés** — contraseña, PIN
- **Algo que tenés** — el teléfono, una llave de seguridad
- **Algo que sos** — huella, cara

MFA (*Multi-Factor Authentication*) es combinar factores **de categorías distintas**. Dos contraseñas no son MFA.

Passkeys

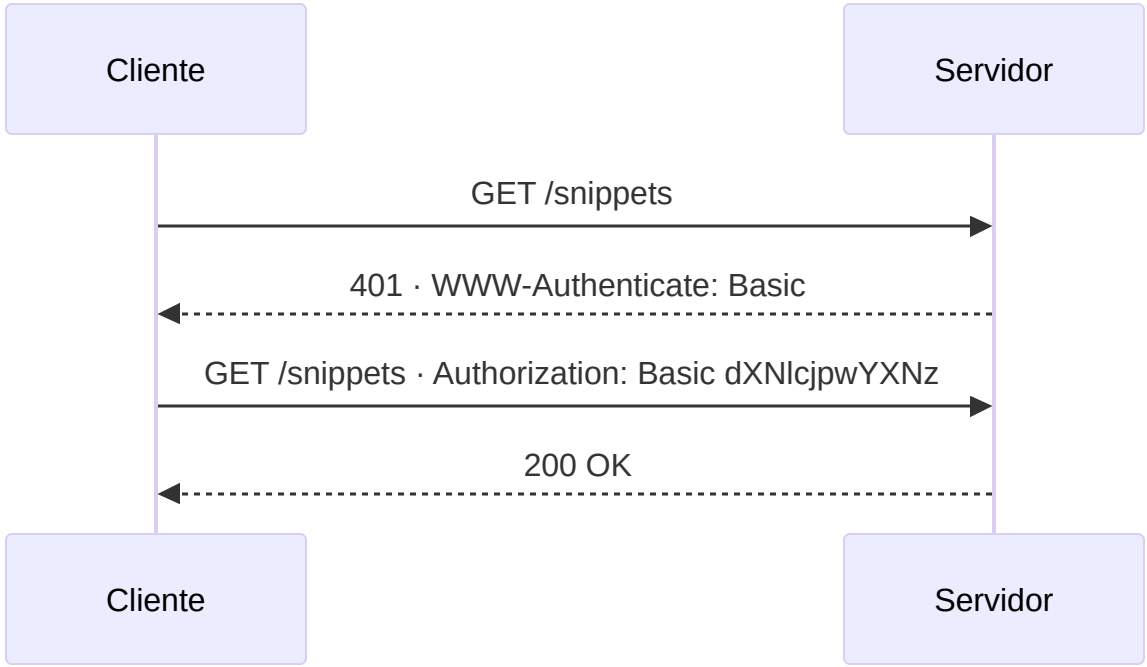


El servidor **no guarda ningún secreto**: no hay nada que filtrar ni que *phishear*.

Ya hiciste *login*. ¿Y en la próxima *request*?

HTTP no tiene memoria. Hay tres formas de que el servidor te reconozca

Opción 1: las credenciales en cada *request*



- Es **HTTP Basic** (RFC 7617). `dXNlcjpwYXNz` es `user:pass` en *base64*: **codificado, no cifrado**
- La contraseña viaja en **cada** *request* y no hay *logout*

Ese *header* `Authorization`

La opción 1 usó `Authorization: Basic`. Es el lugar estándar de HTTP para mandar credenciales:

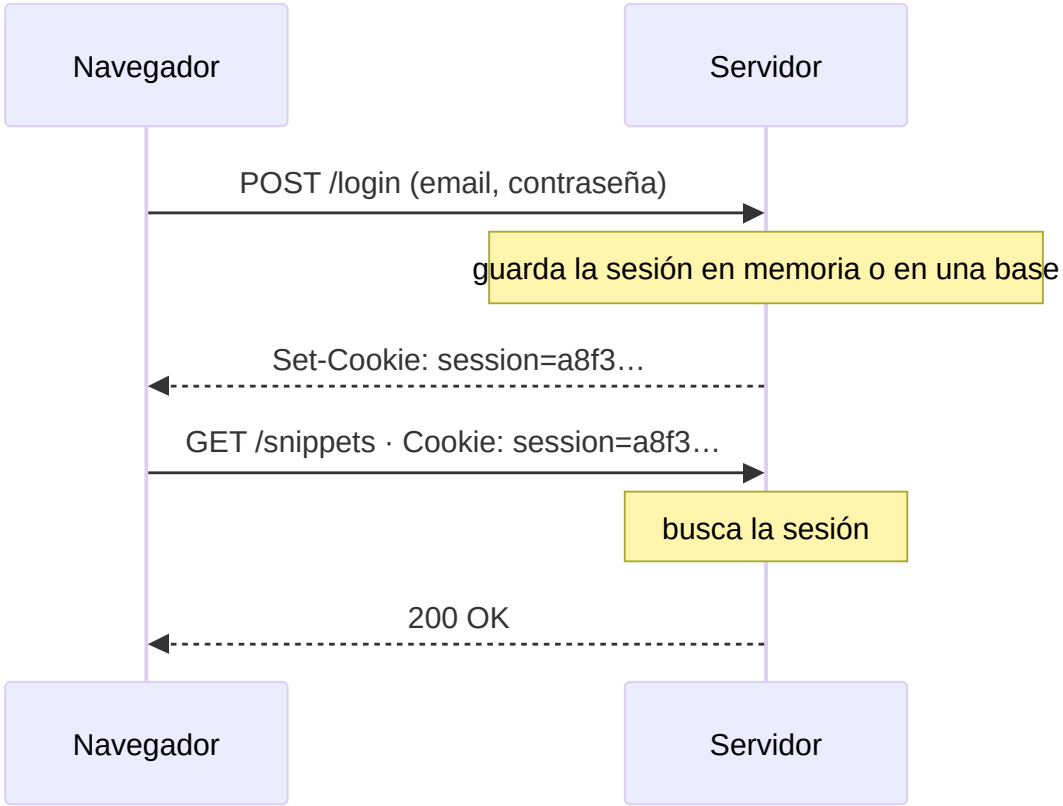
```
Authorization: <esquema> <credenciales>
```

El **esquema** dice cómo leer lo que viene después. ¿Cuántos hay? Sólo registrados, **14**:

Basic RFC 7617	**Bearer** RFC 6750	Negotiate RFC 4559	Digest RFC 7616	DPoP RFC 9449
OAuth RFC 5849	SCRAM-SHA-1 RFC 7804	SCRAM-SHA-256 RFC 7804	Mutual RFC 8120	HOBA RFC 7486
GNAP RFC 9635	PrivateToken RFC 9577	Concealed RFC 9729	vapid RFC 8292	

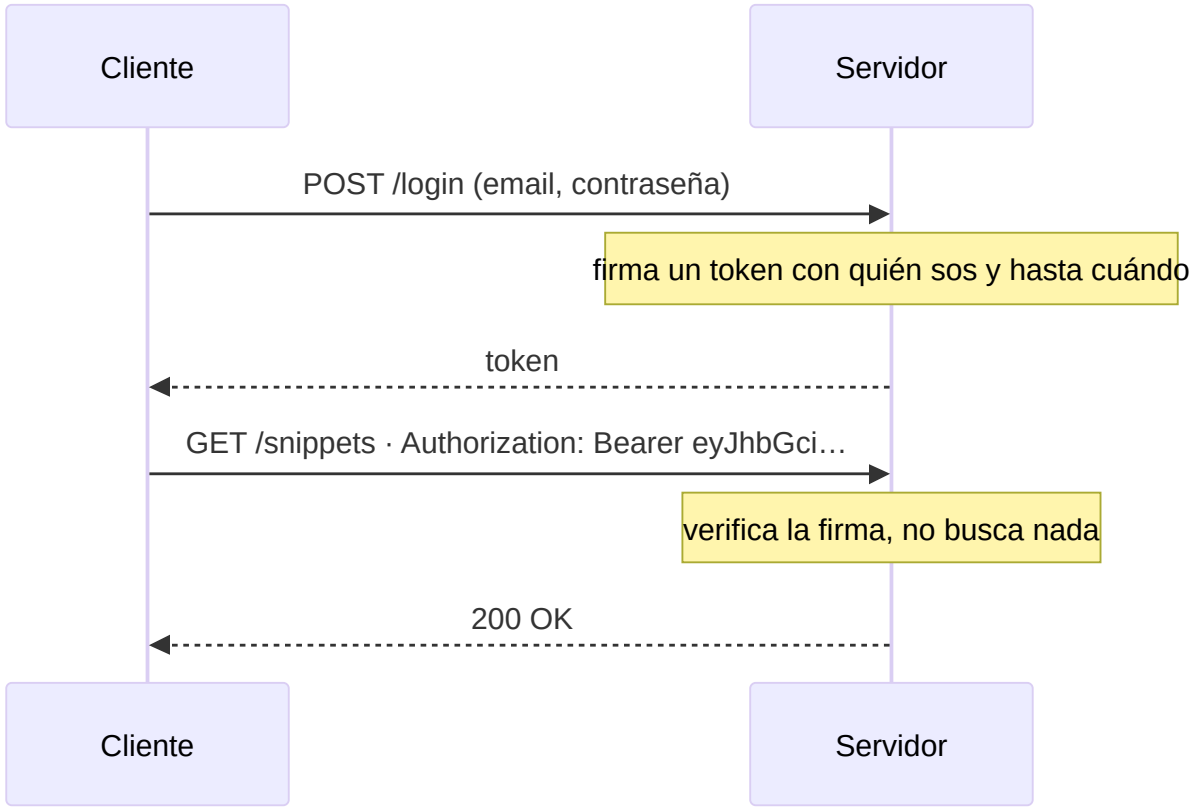
En azul, los dos que vemos hoy. El resto sigue vigente en nichos: Negotiate en redes corporativas, vapid en notificaciones *push*, Digest en cámaras y *routers*.

Opción 2: sesiones (*stateful*)



El estado vive **en el servidor**. El navegador manda la *cookie* solo, en cada *request*.

Opción 3: *tokens (stateless)*



El estado viaja **en el *token***. El servidor sólo verifica que nadie lo tocó.

Va en `Authorization: Bearer`: el esquema *Bearer* (RFC 6750).

Un JWT por dentro

`header.payload.signature` — cada parte en *base64url*

```
{
  "iss": "https://dev-ingsis.us.auth0.com/",
  "sub": "auth0|6512a3f0c9",
  "aud": "https://snippet-searcher/api",
  "exp": 1760389200,
  "scope": "read:snippets write:snippets"
}
```

- **Firmado, no cifrado**: cualquiera lo lee, nadie lo modifica sin romper la firma
- No pongan nada secreto en el *payload*
- Peguen uno en jwt.io para ver qué trae

Sesiones vs *tokens*

	Sesiones	*Tokens*
Escalar entre réplicas	hace falta estado compartido	**cualquier réplica valida**
Revocar acceso ya	**se borra la sesión**	hay que esperar que venza
Que lo emita un tercero	difícil	**natural**
En un monolito	**lo más simple**	más piezas

No hay ganador: **depende de dónde quieren pagar.**

¿Qué tan difícil puede ser un *login*?

Volvamos a la propuesta

```
fun login(request: LoginRequest): Response {
    val user = users.findByEmail(request.email) ?: return notFound("Usuario no existe")
    return if (user.password == request.password) ok() else unauthorized("Contraseña incorrecta")
}
```

¿Qué puede salir mal?

Guardar contraseñas

- **En claro**: una filtración de la base las expone todas, y la gente las reusa
- **Con un *hash* rápido** (MD5, SHA-256): una GPU prueba miles de millones por segundo
- **Sin *salt**:** contraseñas iguales dan el mismo *hash* y sirven las *rainbow tables*
- **Lo correcto**: una función de *hashing* hecha para contraseñas, **costosa a propósito** y con *salt* — Argon2id, scrypt, bcrypt

* *Salt*: un valor aleatorio, distinto para cada usuario, que se mezcla con la contraseña antes de calcular el *hash* y se guarda al lado de él (no es secreto). Así, dos usuarios con la misma contraseña tienen *hashes* distintos, y las *rainbow tables* (tablas precalculadas de «contraseña → *hash*») dejan de servir: hay que atacar cada *hash* por separado.

Atacar el *login*

- ***Credential stuffing***: probar pares email/contraseña filtrados de otros sitios
- **Fuerza bruta**: hace falta *rate limiting* y bloqueo, sin que eso sea un DoS contra el usuario
- **Enumeración**: «usuario no existe» vs «contraseña incorrecta» le dice al atacante qué emails existen
- ***Timing attacks***: `==` corta en el primer carácter distinto; hay que comparar en tiempo constante

Después del *login*

- **Recupero de contraseña**: el flujo más atacado — *tokens* de un uso, que venzan, sin revelar si el email existe
- **Sesiones**: *session fixation*, expiración, un *logout* que de verdad cierre
- **CSRF**: si la sesión va en una *cookie*, otro sitio puede hacer que el navegador la mande
- **XSS**: si el *token* va en `localStorage`, cualquier *script* inyectado lo lee

Si usan JWT

- `alg: none`: librerías que aceptaban *tokens* sin firma porque el *header* lo pedía
- **No validar** `iss`, `aud` **o** `exp`: aceptar *tokens* de otro emisor, para otra API, o vencidos
- **No se puede revocar**: un *token* robado vale hasta que vence
- **Confusión de algoritmo**: verificar un RS256 como si fuera HS256 con la clave pública

Y además, lo que el usuario espera

- MFA y *passkeys*
- *Login* con Google o GitHub
- Verificar el email
- Bloquear una cuenta comprometida
- Auditoría: quién entró, cuándo y desde dónde

Cada una es un *feature* **que alguien tiene que construir y mantener.**

La autenticación no se implementa.

Se delega.

Autorización

¿Quién puede hacer qué?

La autorización sí es (en parte) de ustedes

El proveedor de identidad sabe **quién es** el usuario.
No sabe **de quién es** cada *snippet* ni **con quién** se compartió.

Eso es dominio. La pregunta a responder: **¿quién puede leer, editar y borrar un *snippet*?**

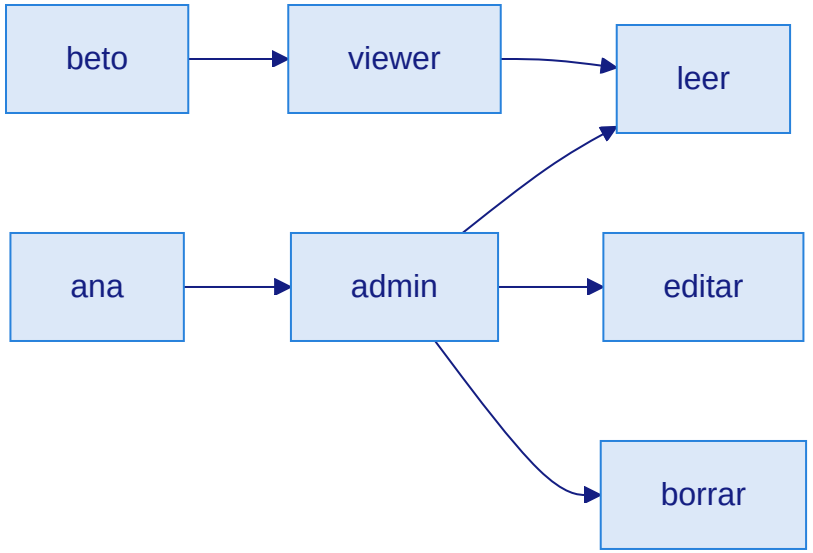
ACL — *Access Control List*

Cada recurso tiene su lista de quién puede qué.

`snippet-42`	leer	editar	borrar
ana	✓	✓	✓
beto	✓		

- **Bien**: simple, directo, fácil de entender
- **Mal**: miles de recursos × miles de usuarios = miles de listas que mantener

RBAC — *Role-Based Access Control*



- **Bien**: el más común; los permisos se administran por rol, no por persona
- **Mal**: no expresa «sólo **mis** *snippets*» — el permiso depende del recurso

ABAC — *Attribute-Based Access Control*

Reglas sobre atributos del usuario, del recurso y del contexto:

```
permitir editar si
     recurso.dueño == usuario.id
  o (usuario.equipo == recurso.equipo y es horario laboral)
```

- **Bien**: muy expresivo
- **Mal**: difícil de auditar — «¿quién puede ver este *snippet*?» no tiene respuesta fácil

Comparando

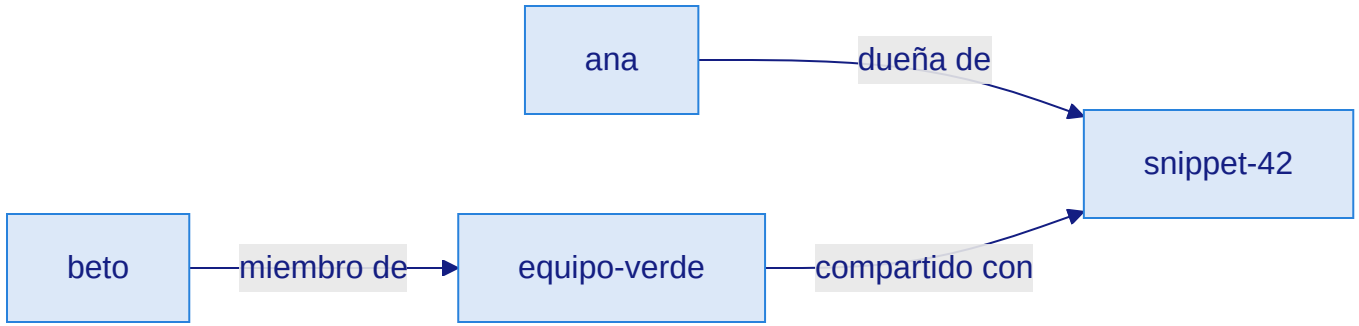
	Decide por	Bien para	Se complica cuando
ACL	lista por recurso	pocos recursos, compartir puntual	crece la escala
RBAC	rol del usuario	permisos por tipo de usuario	importa **cuál** recurso
ABAC	atributos y contexto	reglas finas y condicionales	hay que auditar

En la práctica se combinan: **RBAC para la app, ACL para los recursos.**

«El dueño edita y borra; con quien se compartió, lee.»

¿Qué esquema usarían en el Snippet Searcher?

Curiosidad: ReBAC — *Relationship-Based Access Control*



beto puede leer `snippet-42` porque hay un camino de relaciones.

Es como Google resuelve los permisos de Drive y YouTube (**Zanzibar**).

Scopes ≠ permisos de dominio

`read:snippets` dice que **la aplicación** puede leer *snippets* en nombre del usuario.

No dice **cuál** *snippet*.

- El *scope* lo chequea el filtro de seguridad
- «¿Es dueño o se lo compartieron?» lo chequea **su código**

OAuth 2.0

El problema original

Alrededor de 2007, para que Facebook o Twitter encontraran a tus contactos de Gmail **les dabas tu contraseña de Gmail**.

- Acceso total a la cuenta, no sólo a los contactos
- Para revocarlo, cambiar la contraseña
- La app la guardaba quién sabe cómo

¿Cómo le das a una app acceso **a una parte** de tu cuenta, sin darle la llave entera?

¿Qué es OAuth 2.0?

Un **estándar** (RFC 6749, 2012) que define **el protocolo** con el que una aplicación **obtiene un permiso acotado para usar una API en nombre de un usuario**, sin conocer su contraseña.

- El usuario se autentica **con el servicio**, no con la app
- El usuario **consiente** qué puede hacer la app: los *scopes*
- La app recibe un ***access token***: acotado, que vence y se puede revocar

Es **autorización delegada**: el usuario le delega a la app un permiso suyo.

Lo usan **Google, GitHub, Microsoft, Slack y Spotify**, entre muchos otros: es la pantalla de «*X* quiere acceder a tu cuenta».

Qué define el estándar (y qué no)

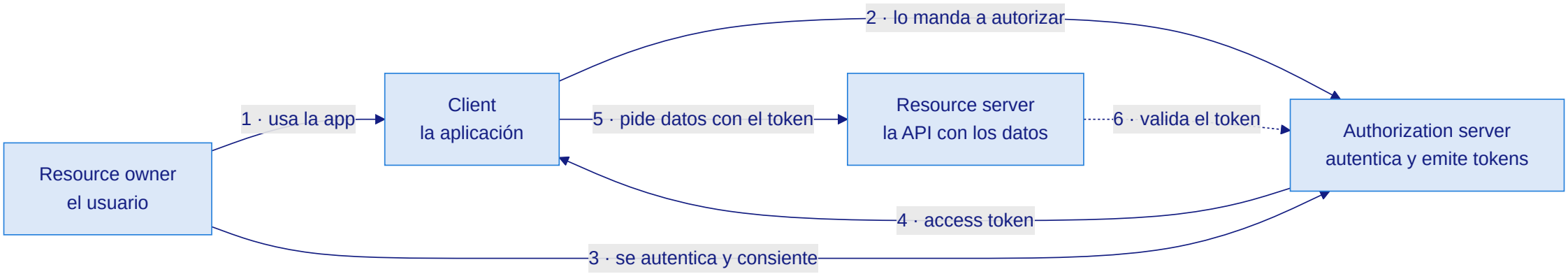
Define

- **Los cuatro roles**: quién pide, quién autoriza y quién tiene los datos
- **Dos *endpoints*** del *authorization server*: `/authorize`, donde el usuario se loguea y consiente, y `/token`, donde la app canjea y renueva
- **Los *grant types***: las formas de conseguir un *token*
- **Los parámetros y las respuestas**: `client_id`, `redirect_uri`, `scope`, `grant_type`, `access_token`, `expires_in`, errores como `invalid_grant`
- **Dos tipos de *client***: confidenciales (guardan un secreto) y públicos (no pueden)

No define

- **Cómo se autentica el usuario** (contraseña, MFA, *passkey*) → lo decide el *authorization server*
- **El formato del *token*** → JWT es otro estándar (RFC 7519)
- **Cómo se manda a la API** → `Authorization: Bearer` (RFC 6750)
- **PKCE** → se agregó después (RFC 7636)

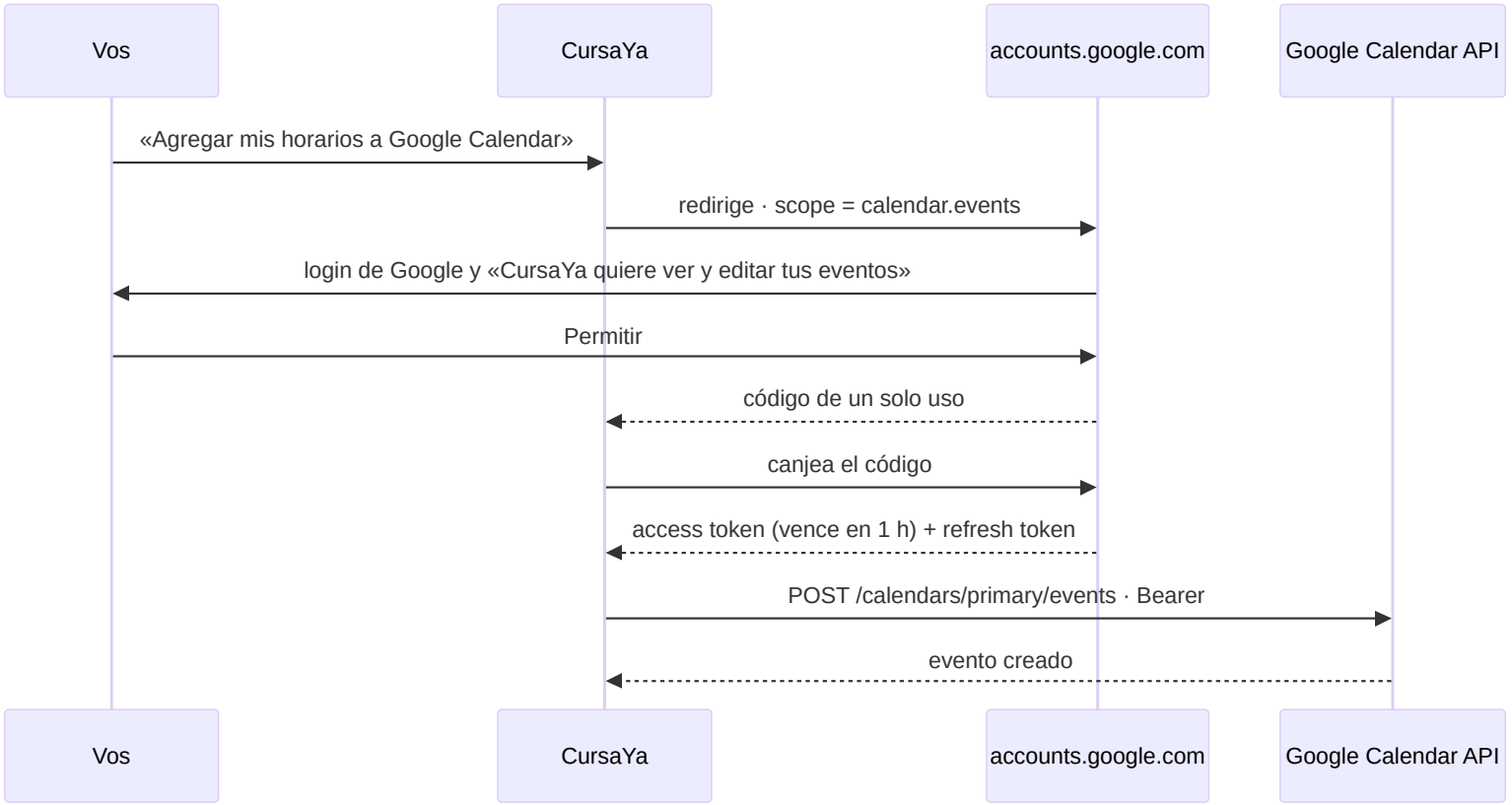
Los cuatro roles



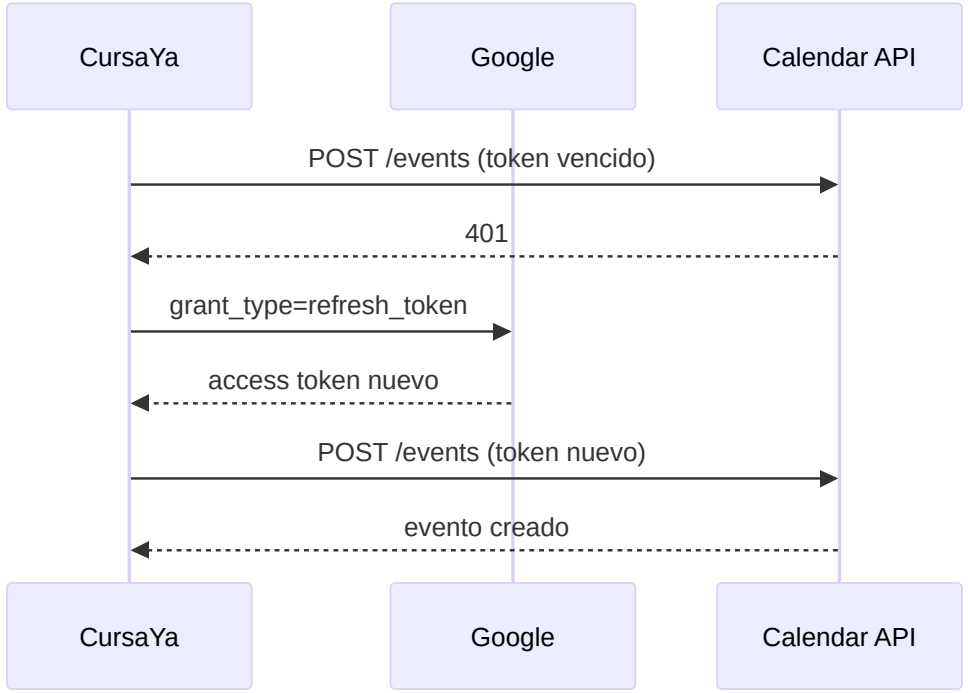
El *client* **nunca ve la contraseña**: sólo recibe el *token*.

Un ejemplo concreto

CursaYa quiere agregar tus horarios de cursada a tu Google Calendar.



¿Y cuando vence el *token*?

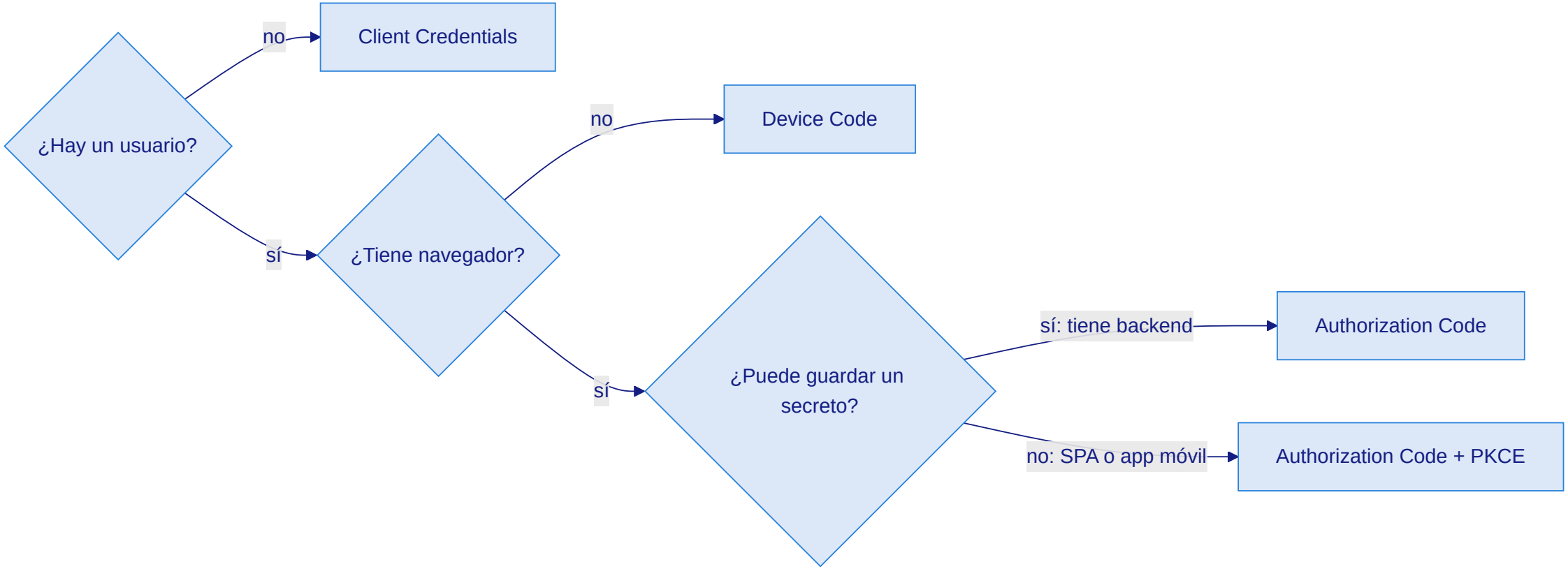


- **No te vuelve a preguntar nada**: el permiso sigue vigente
- El *refresh token* vive **en el *backend*** de CursaYa, nunca en el navegador
- Para usarlo, CursaYa manda también **sus propias credenciales**
- Si revocás el acceso, deja de servir

Los flujos (*grant types*)

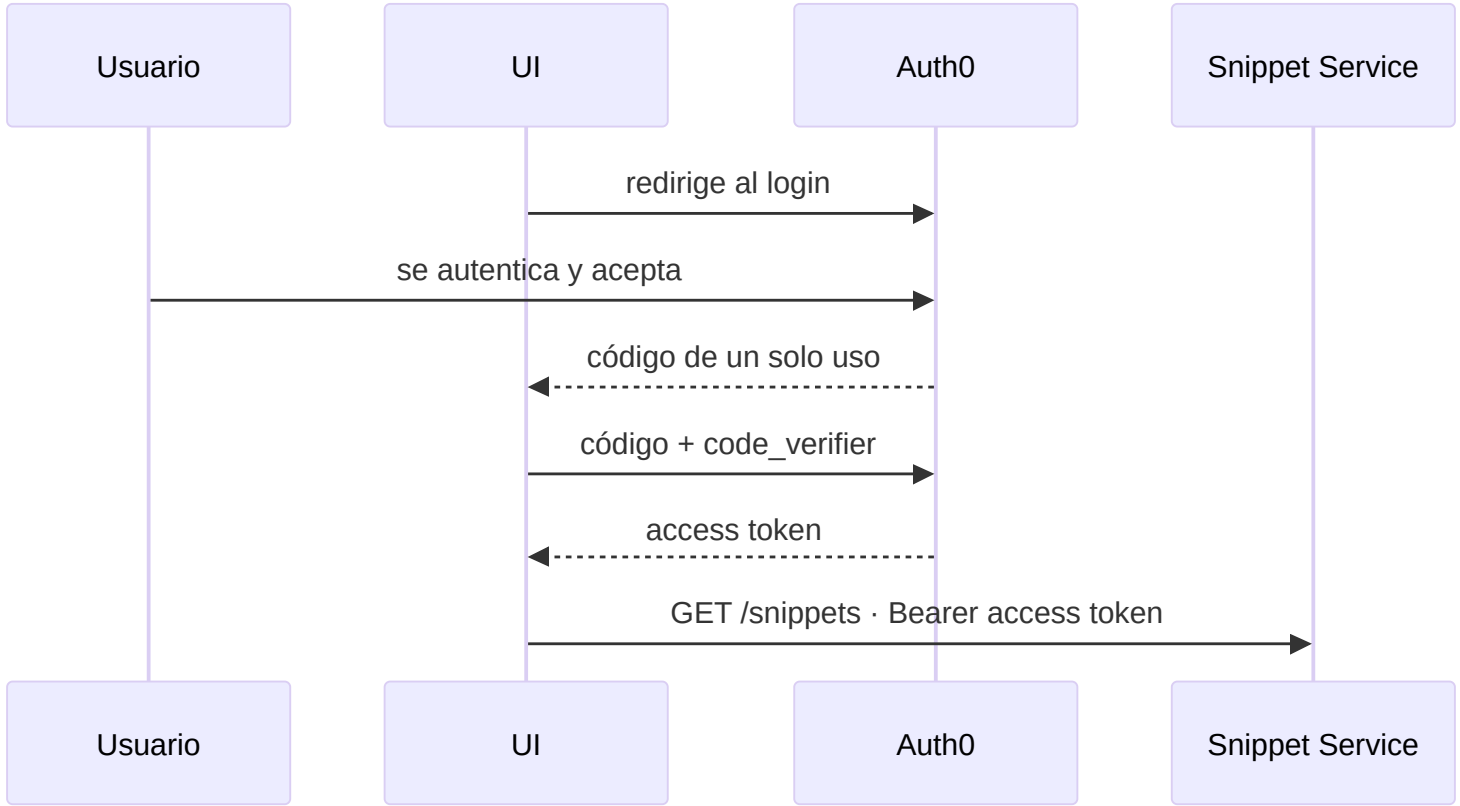
Flujo	Para qué	Ejemplo
Authorization Code (+ PKCE)	una app **con usuario** que tiene navegador	CursaYa con Google Calendar · la UI del TP
Client Credentials	**sin usuario**: un servicio habla con otra API	su *backend* con la *Management API* de Auth0
Device Code	dispositivos **sin navegador ni teclado**	loguearse en Netflix en la tele con un código desde el celular
Refresh Token	conseguir un *access token* nuevo **sin volver a preguntar**	CursaYa al cuatrimestre siguiente
~~*Password* (ROPC)~~	la app pide la contraseña y la manda al servidor · **eliminado en OAuth 2.1**	apps propias con su propio formulario de *login*
~~*Implicit*~~	SPAs sin *backend*, antes de PKCE: el *token* vuelve en la URL · **eliminado en OAuth 2.1**	las SPAs que hacían *login* desde el navegador, antes de PKCE

¿Cuál uso?



Y en todos, ***Refresh Token*** para renovar sin volver a preguntar.

En el Snippet Searcher: *Authorization Code* + PKCE



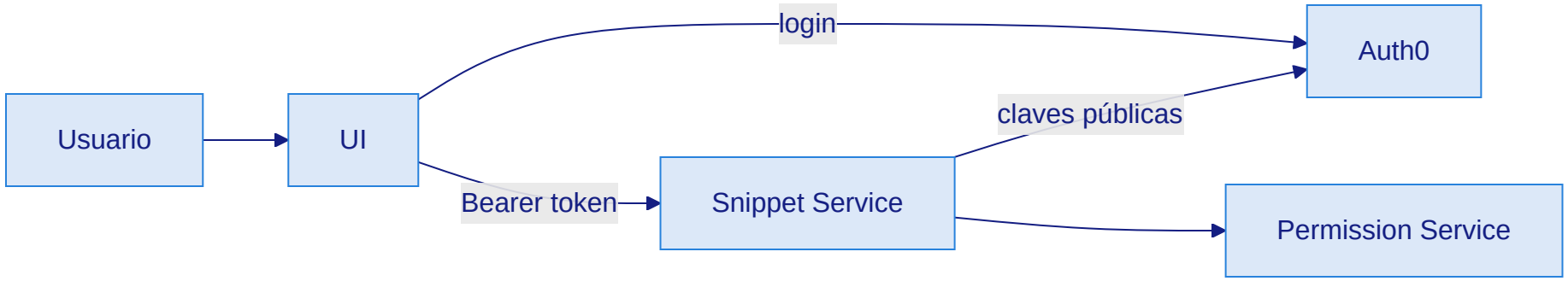
Mismos cuatro roles: el usuario, la UI, **Auth0** y **sus servicios**. La contraseña **sólo la ve Auth0**.

OAuth 2.0 en Auth0

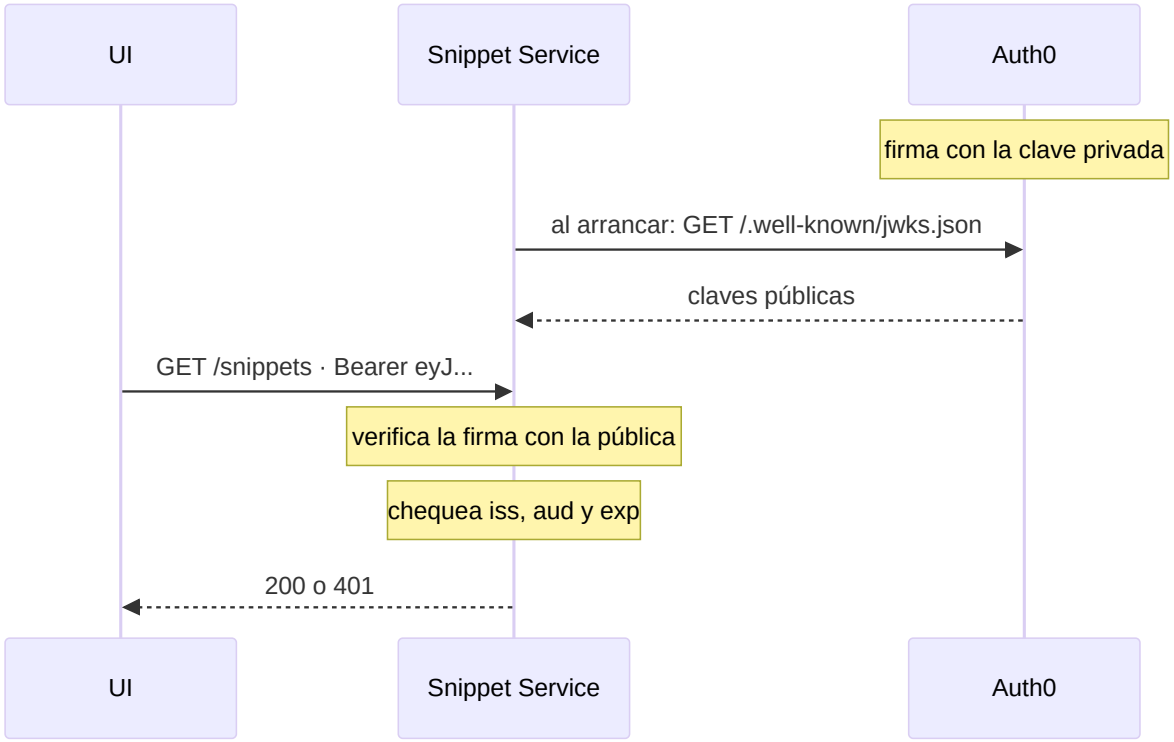
OAuth 2.0	En Auth0	En la demo
Resource owner	*User*	el usuario de prueba
Client	*Application* (*SPA*, *Regular Web App*, *M2M*)	la app que pide el *token* (`client_id`)
Authorization server	el *tenant*: `dev-xxxx.us.auth0.com`	`AUTH_SERVER_URI` → `issuer-uri`
Resource server	*API*	el servicio de Spring Boot
Audience (`aud`)	el *Identifier* de la *API*	`AUTH0_AUDIENCE` → `AudienceValidator`
Scopes	*Permissions* de la *API*	`read:snippets`
Access token	un JWT firmado por el *tenant*	lo que valida el `jwtDecoder`

Qué delegar y qué es de ustedes

El Snippet Searcher con Auth0



Cómo valida el *token* un servicio



- La **privada firma** y nunca sale de Auth0
- La **pública verifica** y la puede bajar cualquiera: no sirve para firmar
- **No le pregunta nada a Auth0 en cada *request***: valida solo
- Si alguien cambia un byte del *payload*, la firma no coincide: **401**

La línea

Delegado

- *Login* y registro
- Guardar contraseñas
- MFA, *passkeys*, *login* social
- Recupero de cuenta
- Emitir *tokens*

De ustedes

- Validar el *token* en **cada** servicio: firma, `iss`, `aud`, `exp`
- Chequear el *scope*
- La autorización de dominio: quién puede qué sobre cada *snippet*

Proveedores de identidad

- **Auth0** / **Okta** — SaaS
- **Keycloak** — *open source*, se hostea uno mismo
- **AWS Cognito**, **Firebase Authentication**, **Microsoft Entra ID**

Todos hablan **OAuth 2.0**: el código del *resource server* es el mismo.

Demo

auth0-integration

La demo

1. En Auth0: la API, la *application*, un usuario y el permiso `read:snippets`
2. `GET /snippets` sin *token* → **401**
3. Leer el *token* en jwt.io
4. `GET /snippets` con el *token* → **200**
5. `GET /scopedAuthenticate` con y sin el *scope* → **403** vs **200**

Lo que valida el servicio

```
@Bean
fun jwtDecoder(): JwtDecoder {
    val decoder = NimbusJwtDecoder.withIssuerLocation(issuer).build()
    decoder.setJwtValidator(
        DelegatingOAuth2TokenValidator(
            JwtValidators.createDefaultWithIssuer(issuer),  // firma, iss, exp
            AudienceValidator(audience),                    // aud
        )
    )
    return decoder
}
```

Es **todo** lo que el servicio sabe de autenticación.

Referencias

- RFC 9110 — *HTTP Semantics* · RFC 7617 — *Basic Authentication* · RFC 7519 — *JWT*
- RFC 6749 — *OAuth 2.0* · RFC 7636 — *PKCE* · *OAuth 2.1* (IETF draft)
- W3C *Web Authentication*
- NIST SP 800-63B (autenticación) · NIST SP 800-162 (ABAC)
- Sandhu et al., «*Role-Based Access Control Models*», IEEE Computer, 1996
- Pang et al., «*Zanzibar: Google's Consistent, Global Authorization System*», USENIX ATC 2019
- OWASP Cheat Sheet Series — *Authentication*, *Password Storage*, *JSON Web Token*
- github.com/austral-ingsis/auth0-integration